

Power Platform Architecture Guidelines

Architecture Discovery and Blueprinting Guidelines for Solution Architects
July 2026

Table of Contents

Table of Contents.....2

1. Purpose & Overview2

2. Solution Discovery for Power Platform Architecture4

3. Required Inputs for a Solution Blueprint.....4

4. Handling Partial Inputs & Assumptions.....7

5. Solution Blueprint Structure.....7

6. Guidelines for Blueprint Content.....8

7. Blueprint Quality Checklist12

1. Purpose & Overview

This document guides solution architects through early-stage discovery and high-level architectural design for Microsoft Power Platform solutions. It defines the inputs to gather,

decisions to document, assumptions to validate, and blueprint sections to produce before detailed implementation begins. The guidance can also support AI-assisted discovery and draft blueprint generation, provided that all outputs are reviewed and validated by a qualified solution architect.

The goal is to help the solution architect structure stakeholder conversations, evaluate architectural trade-offs, and produce a clear Solution Blueprint that can be refined through validation. This document focuses on architecture discovery and blueprinting; it does not replace detailed design, implementation planning, security review, licensing analysis, or formal governance approval.

1.1 Architecture Accountability

AI-assisted outputs must be treated as drafts. The solution architect remains accountable for validating assumptions, assessing trade-offs, confirming constraints, and approving architectural recommendations before they are used as guidance for delivery or implementation.

1.2 How to Use This Document

Human architects should use this document to structure discovery, validate design decisions, and produce consistent Solution Blueprints. AI-assisted workflows may use it to ask clarifying questions, organize inputs, draft blueprint sections, and flag risks. In all cases, unresolved assumptions, missing evidence, and material risks must be explicitly identified for human review.

1.3 Solution and Workload Terminology

Power Platform practitioners commonly use the term solution to describe the set of apps, automations, data sources, integrations, security controls, and lifecycle practices that deliver a business outcome. Microsoft Power Platform Well-Architected guidance uses the term workload for the same architectural scope. In this document, solution and workload are treated as equivalent when describing the architecture to be discovered, evaluated, and documented. The term workload may be used when alignment with Well-Architected guidance is important.

1.4 Apply Guidance Proportionally

The solution architect should apply this guidance according to the workload's complexity, size, business impact, data sensitivity, user scale, integration criticality, operational risk, and regulatory or financial exposure. Lightweight solutions may require a concise blueprint, while business-critical or highly integrated workloads require deeper analysis, stronger validation, and clearer documentation of risks and decision rationale.

- **Microsoft Power Platform Well-Architected Framework**
- **Power Platform Adoption Guidance**
- **Power Platform Admin Documentation** (prevails in case of conflicting detail)

The blueprint process helps solution architects make informed design decisions aligned to the five Power Platform Well-Architected pillars: Reliability, Security, Operational Excellence, Performance Efficiency, and Experience Optimization.

- Reliability
- Security
- Operational Excellence
- Performance Efficiency
- Experience Optimization

Early solution discovery often involves uncertainty. This guideline emphasizes capturing assumptions and progressively refining details as more information becomes available.

2. Solution Discovery for Power Platform Architecture

During discovery, the solution architect must collect enough business, technical, operational, and governance context to produce an initial architecture. The solution architect should use a structured approach throughout the discovery process:

1. **Confirm Understanding** – Restate the problem scenario and objectives to ensure alignment with the stakeholders.
2. **Identify Key Input Areas** – Structure the discussion around major categories (business, users, data, etc.) to systematically cover all aspects of the solution.
3. **Ask Targeted Questions** – For each category, ask clear questions that elicit detailed requirements or constraints.
4. **Allow Progressive Elaboration** – If some information is unavailable, proceed iteratively only when doing so is safe. The solution architect may produce a partial blueprint, but must clearly label assumptions, unresolved questions, and validation needs.
5. **Summarize & Validate** – Periodically summarize gathered inputs back to stakeholders for validation and catch any misunderstandings early.

Throughout discovery, the solution architect must help stakeholders articulate business goals, success criteria, constraints, and decision drivers instead of collecting only technical specifications.

The solution architect must not invent missing business constraints, licensing conditions, security requirements, data classifications, or integration behavior. When information is missing, the blueprint must distinguish confirmed facts, assumptions, risks, recommendations, and open questions.

3. Required Inputs for a Solution Blueprint

To create a useful Power Platform Solution Blueprint, the solution architect should gather the following input categories and adapt the guiding questions to the engagement context. Each input should be supported by evidence when available, not only by verbal assumptions.

3.1 Business Context & Goals

- **Purpose:** Business problem, expected value, adoption outcomes, and measurable success criteria for the solution or workload.

- **Guiding Questions:** "What business problem or opportunity does this solution address? What value should it deliver? What are the primary objectives, success metrics, baseline measures, adoption indicators, and business owners or sponsors? How will value be reviewed after delivery?"
- **Why It Matters:** Understanding the business context ensures the solution aligns with strategic goals and measurable outcomes.
- **Expected Evidence:** Business case, process overview, KPI definitions, baseline metrics, adoption goals, stakeholder goals, pain points, current-state limitations, target operating model, value hypothesis, or review cadence.

3.2 Functional Scope & Requirements

- **Purpose:** Key capabilities and processes the solution must support.
- **Guiding Questions:** "Which business processes or tasks will this solution enable or improve? What are the essential features or user stories?"
- **Why It Matters:** Clarifying the scope prevents feature creep and guides architecture choices, including required components from the start.
- **Expected Evidence:** User stories, process maps, backlog items, acceptance criteria, screenshots, forms, approval matrices, or current manual procedures.

3.3 Users & Personas

- **Purpose:** The intended users, their roles, and usage scenarios.
- **Guiding Questions:** "Who are the primary users or personas? What are their technical skill levels and devices? How many users (concurrent/total) are expected, and how frequently will they use the solution?"
- **Why It Matters:** User profiles influence UI decisions (Canvas vs. Model-driven), authentication method (internal vs. external users), and capacity planning for scale.
- **Expected Evidence:** Persona definitions, user volumes, access groups, device profiles, accessibility needs, adoption constraints, and support model expectations.

3.4 Data Sources & Integrations

- **Purpose:** What data the solution will consume or produce, and how it connects with other systems.
- **Guiding Questions:** "What are the data sources and systems involved? Are we using Dataverse, external databases, SharePoint, or third-party APIs? Will data migration from legacy systems be needed? What integrations are required, and are they real-time, scheduled, event-driven, or manual?"
- **Why It Matters:** Knowing data location and integration needs informs component selection (e.g., Dataverse vs. SharePoint), connector usage, and potential integration patterns.
- **Expected Evidence:** Data model samples, source system inventory, connector requirements, API documentation, integration ownership, data quality constraints, and migration scope.

3.5 Security & Compliance Requirements

- **Purpose:** Constraints or policies around security, privacy, and compliance.

- **Guiding Questions:** "Is there any sensitive data (PII, confidential) that affects design? What compliance standards apply (GDPR, HIPAA, etc.)? Are there data residency requirements or specific encryption needs? Who should have which permissions or roles?"
- **Why It Matters:** Early identification of security needs ensures the solution includes proper role-based access controls and compliance measures. It may also determine environment location for data residency and use of features like field security or encryption.
- **Expected Evidence:** Data classification, compliance obligations, security policies, DLP policies, retention requirements, audit requirements, identity model, and least-privilege access rules.

3.6 Performance & Scalability Considerations

- **Purpose:** Volume, scale, and performance expectations.
- **Guiding Questions:** "How much data (records per day, total size) will the solution handle? How many concurrent users at peak? Are there critical performance requirements (response time, throughput)? Any seasonal or usage growth expectations?"
- **Why It Matters:** This helps the blueprint include plans for performance (delegating queries, caching) and scaling (partitioning data or offloading tasks to other services). It ties directly into the Well-Architected Performance Efficiency and Reliability pillars.
- **Expected Evidence:** Estimated transaction volumes, peak concurrency, data growth projections, response-time expectations, dependency limits, and known performance pain points.

3.7 Environment & ALM/Governance Constraints

- **Purpose:** Operational and lifecycle management requirements.
- **Guiding Questions:** "Will this be built in an existing environment or a new one? What organizational rules apply to environments, DLP, ALM, pipelines, managed environments, support, monitoring, and security review? Is there a CoE, platform administration team, or governance team that must provide guidance or approval?"
- **Why It Matters:** Environment, ALM, monitoring, availability, and governance constraints often come from organizational standards. The blueprint should identify applicable rules, dependencies, and approval needs, but should not redefine platform policies that are owned by the CoE, platform administration, security, or governance teams.
- **Expected Evidence:** Existing environment strategy, ALM standards, CoE guidance, DLP policies, managed environment requirements, pipeline standards, architecture review process, naming conventions, ownership model, and platform administration constraints.

3.8 Licensing, Capacity & Feature Availability

Purpose: Licensing, capacity, and feature constraints that affect feasibility, cost, component selection, and delivery approach.

Guiding Questions: "Which Power Platform licenses are available? Which licenses or add-ons are required? Will the solution use premium connectors, Dataverse, AI Builder, Copilot capabilities, Power Pages, managed environments, or other features with licensing

implications? Are there Dataverse capacity, API request, storage, or cost constraints to consider? Who owns licensing and capacity decisions?"

Why It Matters: Licensing and feature availability can determine whether a recommended architecture is viable. The blueprint should identify required capabilities and flag gaps for validation with licensing, procurement, platform administration, or CoE teams.

Expected Evidence: License inventory, entitlement assumptions, cost ownership, capacity reports, premium connector requirements, environment feature availability, managed environment eligibility, and known licensing restrictions.

4. Handling Partial Inputs & Assumptions

In initial discovery, some details may be unknown or evolving. The following approach ensures the blueprint process remains productive even with incomplete information:

- **Prompt for Missing Info:** If an important input such as primary users, data volume, licensing, or governance constraints is missing, the solution architect should ask clarifying questions before proceeding.
- **Make and Mark Assumptions:** If progress must continue without certain inputs, proceed with reasonable assumptions, explicitly noting them. For example: "Assumption: The solution will initially serve ~50 users, since no user count was provided."
- **Highlight Risks:** Each assumption should be accompanied by a note on potential risk or impact if the assumption is wrong. For example: "If actual usage exceeds this assumption, additional performance tuning may be required."
- **Revisit & Refine:** The solution architect should track outstanding questions and assumptions and revisit them later. A dedicated "Assumptions & Risks" section in the output blueprint should capture these items so reviewers know what to verify with stakeholders, the CoE, platform administration, security, or governance teams.

5. Solution Blueprint Structure

After gathering inputs, the solution architect produces or validates a Solution Blueprint — a structured document capturing the high-level architecture of the proposed Power Platform solution or workload. The table below outlines the standard sections and their purposes:

Blueprint Section	Purpose & Content
Solution Overview & Objectives	Summarize the business context, problem statement, and key objectives or success metrics. Sets the stage for why the solution is needed.
Architecture Overview & Components	Outline the chosen Power Platform components (e.g., Canvas App, Model-Driven App, Cloud Flow, Dataverse) and how they fit together. Identify major solution elements and their roles.
Data & Integration Approach	Describe how data will be stored and managed and any key integration points with external systems. List critical connectors or integration patterns.
Security & Compliance	State how the solution addresses security (user access roles, environment security) and compliance needs (data privacy, retention, DLP considerations).
ALM & Deployment Strategy	High-level plan for lifecycle and deployment considerations, including environment strategy, managed or unmanaged solution approach, pipeline expectations, and organizational governance dependencies. Reference CoE or platform administration guidance where applicable.
Licensing, Capacity & Feature Constraints	Document available and required licenses, premium features, Dataverse capacity, AI-related capacity, managed environment eligibility, and cost or entitlement assumptions that may affect the architecture.
Architectural Decisions & Rationale	Document important design decisions and why they were made, structured by the Well-Architected pillars: Reliability, Security, Operational Excellence, Performance Efficiency, and Experience Optimization. Each decision should include rationale, trade-offs, assumptions, and the requirement or constraint that drove it.
Governance & Organizational Dependencies	Identify CoE, platform administration, security, compliance, support, monitoring, and availability dependencies that may affect the blueprint. The blueprint should reference organizational rules instead of redefining them.
Assumptions & Risks	List any assumptions made due to missing information, along with potential impacts or risks if they prove false. Alerts architects to areas needing confirmation.
Solution Diagram(s)	Include one or more Mermaid.js diagram code blocks as placeholders for architecture visuals, illustrating component relationships, data flows, and user interactions.

These standard sections ensure each blueprint is comprehensive yet concise. The solution architect should scale the depth of each section according to workload complexity, size, business impact, and risk. Additional sections such as Timeline & Roadmap, Glossary, or Well-Architected Review Notes can be added when needed, but the blueprint should remain focused on architectural understanding rather than detailed implementation planning.

6. Guidelines for Blueprint Content

6.1 Solution Overview & Objectives

Begin the blueprint with an executive-style summary that restates the business problem and desired outcomes in 2–3 paragraphs. The overview should:

- Emphasize the value proposition, value hypothesis, adoption outcomes, and measurable success criteria for the business.
- Mention key stakeholders, business owners, sponsors, or accountable decision makers where relevant.
- Provide context for all readers and identify how success, adoption, or value realization will be validated after delivery.

6.2 Architecture Overview & Components

Provide a narrative of how the solution will work at a high level. When documenting components:

- Identify each major component and use official product names (e.g., "Power Apps (Canvas)", "Power Automate (Cloud Flow)").
- Indicate how components interact, referencing diagrams for clarity.
- Mention additional patterns or services involved (e.g., Azure Functions for complex processing, AI Builder for OCR) with brief rationale.

Use **Power Platform Component & Feature Catalog** guideline document as reference for component selection.

Example: "A Canvas App will serve as the user interface for field technicians, connected to a Dataverse environment for data storage. Power Automate Cloud Flows will handle approvals and data synchronization with an ERP system via a standard connector."

6.3 Data & Integration Approach

Describe decisions around data storage and integration:

- **Primary Data Source:** Identify whether the solution will use Dataverse for relational data, SharePoint for simple lists, or another storage option.
- **Key Entities:** Note important Dataverse entities and relationships central to the solution (e.g., Customer, Order, and Invoice tables).
- **Integration Points:** Summarize external systems the solution will connect to and how integration is achieved (standard connector, custom connector, or Power Query Dataflows).
- **Data Migration:** If data migration is in scope, briefly outline the approach, phases, and cutover strategy.
- **Data Constraints:** Highlight key data-related decisions and their rationale, such as choosing Dataverse for its built-in security and relational capabilities.

6.4 Security & Compliance

Outline the security model and compliance considerations:

- **User Access Control:** Describe how users will be authenticated (e.g., via Azure AD) and how data access will be restricted (e.g., Dataverse security roles). For external users, note the identity provider and web role setup.

- **Governance Requirements:** Reference applicable organizational governance, DLP, environment, connector, and data handling policies. Confirm ownership and exceptions with the CoE, platform administration, security, or governance teams where needed.
- **Compliance Requirements:** Address GDPR, record retention, and auditing requirements through mechanisms such as Dataverse auditing or data retention policies.
- **Specialized Security:** Cover field-level encryption or field security profiles for sensitive data, and note network isolation needs where applicable.

Use **Power Platform Best Practices** guideline document as reference for security & compliance features.

6.5 ALM & Deployment Strategy

Provide a high-level plan for building, testing, and delivering the solution. This section should identify architectural lifecycle factors and organizational dependencies, not prescribe detailed implementation tasks unless they materially affect the architecture.

- **Environment Strategy:** Confirm the number of environments (Dev, Test, Prod) and relevant details (e.g., "Development in a Sandbox environment, then packaged as a managed solution for Production").
- **Deployment Pipeline:** Identify whether Power Platform Pipelines or other CI/CD tools are expected by organizational standards. The blueprint should capture pipeline assumptions, ownership, and approvals without replacing the organization's ALM guidance.
- **Governance & CoE:** Identify whether a CoE, platform administration team, security team, or governance team owns standards that affect the solution. Document required reviews, naming conventions, DLP rules, managed environment expectations, and exception processes when they affect the architecture.
- **Support, Monitoring & Availability:** Identify operational factors that affect the blueprint, such as support ownership, monitoring expectations, incident response, availability requirements, backup or recovery expectations, and whether these are governed by organizational standards.

Use **Power Platform Best Practices** guideline document as reference for ALM & Deployment features.

The solution architect should consult the CoE, platform administration, security, compliance, or support teams when organizational standards determine environment strategy, managed environment usage, DLP rules, pipelines, monitoring, availability, or support responsibilities.

6.6 Licensing, Capacity & Feature Constraints

Document licensing and capacity factors that materially affect the architecture. The blueprint should identify available licenses, required licenses, premium connector usage, Dataverse capacity, AI Builder or Copilot-related capacity where applicable, managed environment eligibility, and any cost or entitlement assumptions that require validation. The solution architect should confirm unresolved licensing or capacity questions with the appropriate licensing, procurement, CoE, or platform administration stakeholders.

6.7 Architectural Decisions & Rationale

This section lists major design decisions and their rationale, organized by Well-Architected pillar when useful. For higher-complexity, business-critical, or high-risk workloads, the solution architect should consider performing or planning a Power Platform Well-Architected Review and referencing findings or follow-up actions in the blueprint.

- **Reliability:** e.g., "Use multiple environments (Dev/Test/Prod) and managed solutions to ensure reliable deployments and rollbacks."
- **Performance Efficiency:** e.g., "Chose Dataverse over SharePoint for data storage to handle expected data volume and support complex queries."
- **Security:** e.g., "Implement role-based security in Dataverse and ensure least-privilege access for all users."
- **Operational Excellence:** e.g., "Utilize pipelines for automated deployments, enabling repeatability and reducing manual errors."
- **Experience Optimization:** e.g., "Selected a responsive Canvas App for a tailored UX, ensuring offline capabilities for field users if needed."

Each decision entry should briefly state the alternatives considered and explain why the chosen approach is better. Tie rationales to the corresponding pillar or business requirement, and include cross-references to relevant best practices or governance guidelines.

Each decision should capture the decision, context, alternatives considered, selected approach, rationale, trade-offs, related requirement or constraint, affected Well-Architected pillar, and validation still required. This level of detail should be proportional to the workload's complexity and impact.

6.8 Assumptions & Risks

Summarize the assumptions made during discovery along with their potential impacts:

- **List Assumptions:** Include all significant assumptions (user counts, licensing availability, feature readiness) along with a brief risk or mitigation note for each.
- **Example:** "Assumption: The organization has an existing Power Platform environment to host this solution. Risk: If not, we must plan for environment setup and additional configuration."
- **Revisit Over Time:** This section should be updated as more information becomes available to remove or update resolved assumptions.
- **Call to Action:** End with a note such as: "These assumptions should be confirmed with the stakeholders to ensure the solution's design remains valid."
- **Classify Gaps:** Classify missing information as a blocking gap, an assumption that allows progress with risk, or a deferred detail that can be refined during later design phases.

6.9 Solution Diagram(s)

Include at least one diagram or diagram placeholder to illustrate the solution's high-level architecture. Diagrams should support architectural understanding for human readers and AI-assisted workflows without turning the blueprint into a detailed implementation specification.

A typical system context diagram shows user roles, apps, flows, data sources, and integrations. For example:

```
flowchart LR
    User([User]) -->|uses| App[Canvas App]
    App -->|writes to| DB[(Dataverse)]
    App -.->|calls| Flow[Power Automate cloud flow]
    Flow -->|integrates with| OnPrem[On-Premises System]
    App -->|embedded AI| Copilot[Copilot Component]
```

Diagrams should be included as placeholders for architecture visuals and can be rendered with any Mermaid-compatible tool. Complex scenarios may require multiple diagrams to show different perspectives of the solution.

6.10 Blueprint Scope Guardrails

The Solution Blueprint should remain an architectural artifact. It should not define detailed Power Fx formulas, flow logic, plug-in designs, table-level schemas, backlog task lists, or operational runbooks unless those details are necessary to explain a major architectural decision. Detailed implementation planning should be handled in later design or delivery artifacts.

7. Blueprint Quality Checklist

Before considering a Solution Blueprint complete, the solution architect should verify the following:

7.1 Business Alignment

- The business problem, objectives, and expected value are clearly stated.
- Success metrics, adoption indicators, or measurable outcomes are identified where available.
- Key stakeholders, sponsors, business owners, or accountable decision makers are identified.

7.2 Discovery Completeness

- Major business, user, data, integration, security, governance, licensing, and operational inputs have been captured.
- Missing information is explicitly identified.
- Assumptions are clearly separated from confirmed facts.

7.3 Architectural Soundness

- The proposed architecture addresses the stated business and technical requirements.
- Major component selections are justified.

- Architectural decisions include rationale, alternatives considered, and relevant trade-offs.
- Design decisions are aligned with the applicable Power Platform Well-Architected pillars.

7.4 Security, Governance & Compliance

- Security, privacy, compliance, and data protection considerations have been evaluated.
- Applicable organizational policies, governance requirements, and dependencies have been identified.
- Required reviews, approvals, or guidance from the CoE, platform administration, security, compliance, or governance teams are documented where applicable.

7.5 Licensing, Capacity & Feature Availability

- Available and required licenses have been identified.
- Premium features, connectors, Dataverse usage, AI capabilities, and capacity requirements have been evaluated.
- Licensing, entitlement, capacity, or feature availability assumptions are explicitly documented.

7.6 Operational Readiness

- Environment strategy, ALM approach, deployment considerations, and operational dependencies are identified.
- Support, monitoring, availability, backup, recovery, or operational ownership expectations are documented where relevant.
- Organizational standards that affect the solution are referenced rather than redefined.

7.7 Risks & Validation

- Significant assumptions are documented.
- Associated risks and impacts are identified.
- Open questions and validation activities are listed.
- Areas requiring stakeholder confirmation are clearly highlighted.

7.8 Blueprint Quality

- The blueprint remains focused on architecture and does not contain unnecessary implementation details.
- Diagrams accurately represent the proposed architecture.
- The level of detail is appropriate for the workload's complexity, size, risk, and business impact.
- The blueprint can be understood by both human reviewers and AI-assisted workflows.

7.9 Final Validation Statement

The solution architect should confirm that the blueprint provides sufficient information to evaluate the proposed architecture, understand the major design decisions, identify material

risks and dependencies, and support subsequent design and delivery activities without prematurely prescribing implementation details.